DB 덤프 파일 가이드

수집을 다시 돌리지 않고, 덤프 파일 하나로 청약 플랫폼 DB를 똑같이 옮기는 방법입니다.

기준일	2026-10-02 (아래 명령어는 실제로 실행해서 확인했습니다)
대상	덤프를 **만드는 사람**(2장)과 **받아서 복원하는 사람**(3장)
환경	Windows + Docker Desktop, 명령어는 **PowerShell** 기준, 프로젝트 폴더에서 실행
소요 시간	만들기 약 15초 · 파일 약 63MB · 복원 약 1분

목차

1 **덤프 파일이란**
왜 필요한지, 무엇이 들어 있는지, 코드(Git)와 무엇이 다른지

2 **덤프 만들기** 보내는 사람
명령어 3줄로 파일 만들기, 전달 방법

3 **덤프 복원하기** 받는 사람
준비 → 복원 → 확인까지 순서대로

4 **복원한 뒤**
자동 수집을 켤지, 데이터를 최신으로 유지하는 방법

5 **자주 생기는 문제**
오류 메시지별 원인과 해결

1. 덤프 파일이란

덤프(dump) 파일은 DB 전체(테이블 구조 + 데이터 + 뷰)를 파일 하나로 내보낸 것입니다. 받는 사람이 이 파일을 자기 DB에 **복원**하면, 만든 사람의 DB와 똑같은 상태가 됩니다.

	Git (코드)	덤프 파일 (데이터)
들어 있는 것	수집 코드, 테이블 구조 정의, 뷰 SQL	테이블 13개의 모든 행 + 가공 레이어(뷰 4개)
공유 방법	`git push` / `git pull`	파일을 드라이브·메신저로 전달
받은 뒤	코드만 바뀜. DB는 그대로	DB가 통째로 바뀜

> **왜 필요한가** — Git에는 코드만 올라갑니다. 코드만 받으면 DB는 비어 있어서 전체 수집을 직접 돌려야 하는데, SH 게시판 약 80분 + LH 전체 기간은 하루 10,000건 호출 한도 때문에 2~3일이 걸립니다. API 활용신청(LH 3종, 청약홈 통계 2종)도 각자 해야 합니다. **덤프를 받으면 이 과정 없이 약 1분이면 됩니다.**

덤프에 들어 있는 것 (`cheongyak_20261002.dump` 기준)

구분	테이블	행
원본	`listings` 공고 × 주택형	126,082
	`historical_results` 경쟁률·당첨가점·특별공급	346,328
	`bankbook_stats` · `subscription_stats` 청약홈 통계	2,041,305 · 2,786
	`lh_notice_details` · `sh_notices`	21,327 · 2,621
	`loan_products` · `loan_rate_options`	280 · 682
추출	`notice_documents` · `lh_rental_houses` · `notice_costs` · `notice_option_costs`	302 · 1,692 · 967 · 219
가공	`listing_features` · `recommendable_listings` · `cutoff_stats` · `competition_stats`	복원할 때 같이 만들어짐

테이블별 설명은 "청약플랫폼_데이터구조.pdf"를 보세요. `user_profiles`(사용자 입력 정보)는 현재 0행이라 개인정보는 들어 있지 않습니다.

> **덤프는 만든 시점의 스냅샷입니다.** 복원한 뒤 수집을 켜지 않으면 데이터가 그날에 멈춰 있습니다. 개발·화면 작업에는 충분하고, 최신 데이터가 필요하면 4장을 보세요.

2. 덤프 만들기 — 보내는 사람

프로젝트 폴더(`cheongyak-platform`)에서 PowerShell을 열고 실행합니다. DB 컨테이너가 켜져 있어야 합니다 (`docker compose up -d db`).

```
# ① DB 컨테이너 안에서 덤프 파일 만들기 (약 15초)
docker compose exec -T db pg_dump -U cheongyak -d cheongyak -Fc -f /tmp/cheongyak.dump

# ② 컨테이너에서 내 PC로 꺼내기 — 파일 이름에 날짜를 붙이면 구분하기 좋습니다
docker compose cp db:/tmp/cheongyak.dump .\cheongyak_20261002.dump

# ③ 컨테이너 안의 임시 파일 지우기
docker compose exec -T db rm /tmp/cheongyak.dump
```

옵션	의미
`-U cheongyak -d cheongyak`	DB 사용자와 DB 이름 (docker-compose.yml에 적힌 값)
`-Fc`	압축된 전용 형식. 1.65GB DB가 약 63MB 파일이 됩니다
`-T`	터미널 없이 실행 (스크립트·PowerShell에서 오류 방지)

만든 뒤 확인

```
Get-Item .\cheongyak_20261002.dump | Select-Object Name, @{n="MB";e={[math]::Round($_.Length/1MB,1)}}
# → 약 63MB면 정상. 1MB보다 작으면 실패한 것이니 ①부터 다시 하세요
```

전달

- 구글 드라이브·팀 메신저 등으로 파일을 보냅니다. **어느 커밋의 코드와 짝이 맞는지**(예: `jiyoon(cheongyakP)` 브랜치, 2026-10-02) 같이 알려 주세요.
- **Git에는 올리지 마세요.** 용량이 크고 갱신할 때마다 저장소가 무거워집니다. 프로젝트 폴더에 두더라도 `.gitignore`에 `*.dump`가 있어 커밋되지 않습니다.

> **지금 덤프는 완전한 상태입니다.** LH 2010-10 ~ 현재 21,327건의 상세·공급정보까지 모두 받은 뒤(2026-10-02) 만든 `cheongyak_20261002.dump`(62.7MB)를 시험용 DB에 복원해서 16개 테이블·뷰의 행 수가 원본과 전부 같은 것을 확인했습니다. 새 덤프는 매일 오전 6시 자동 수집이 끝난 뒤에 만들면 그날 데이터까지 들어갑니다.

3. 덤프 복원하기 — 받는 사람

> **복원하면 내 PC의 기존 `cheongyak` DB는 지워지고 덤프 내용으로 바뀝니다.** 직접 넣은 데이터가 있다면 먼저 2장 방법으로 내 DB의 덤프를 떠 두세요.

3-1. 준비

#	할 일	명령어 / 확인
1	Docker Desktop 실행	트레이의 고래 아이콘이 "running"인지 확인
2	덤프와 짝이 맞는 코드 받기	`git fetch origin` → `git switch "jiyoon(cheongyakP)"` → `git pull` (브랜치 이름에 괄호가 있어 따옴표 필수)
3	덤프 파일을 프로젝트 폴더에 두기	예: `cheongyak-platform\cheongyak_20261002.dump`
4	`backend/.env` 확인	데이터만 볼 거면 API 키는 없어도 됩니다. 파일 자체가 없으면 `backend/.env.example`을 복사해 만드세요

3-2. 복원 (프로젝트 폴더에서 PowerShell)

```
# ① DB 컨테이너만 켜고, DB를 쓰고 있는 backend는 잠시 멈춤
docker compose up -d db
docker compose stop backend

# ② 덤프 파일을 DB 컨테이너 안으로 복사
docker compose cp .\cheongyak_20261002.dump db:/tmp/cheongyak.dump

# ③ 기존 DB를 지우고 빈 DB 만들기
docker compose exec -T db psql -U cheongyak -d postgres -c "DROP DATABASE IF EXISTS cheongyak WITH (FORCE)" -c
"CREATE DATABASE cheongyak"

# ④ 복원 (약 1분. 아무 메시지 없이 끝나면 성공)
docker compose exec -T db pg_restore -U cheongyak -d cheongyak --no-owner /tmp/cheongyak.dump

# ⑤ 임시 파일 지우고 backend 다시 켜기 (--build: 새 코드의 라이브러리 설치)
docker compose exec -T db rm /tmp/cheongyak.dump
docker compose up -d --build backend
```

3-3. 확인

```
docker compose exec -T db psql -U cheongyak -d cheongyak -c "SELECT 'listings' AS t, count(*) FROM listings
UNION ALL SELECT 'historical_results', count(*) FROM historical_results UNION ALL SELECT 'bankbook_stats',
count(*) FROM bankbook_stats UNION ALL SELECT 'recommendable_listings', count(*) FROM recommendable_listings"
```

테이블	2026-10-02 덤프 기준	다르면
listings	126,082	0이거나 "relation does not exist"면 ④ 복원이 실패한 것 → 5장 참고 수치가 조금 다르면 다른 날짜의 덤프입니다 (정상)
historical_results	346,328	
bankbook_stats	2,041,305	
recommendable_listings	841	이 뷰는 "오늘 기준 접수 예정·접수 중"이라 **날짜가 지나면 줄어드는 게 정상**입니다

DBeaver에서는 Host `localhost` · Port `5432` · Database / User / Password `cheongyak`로 연결하고, 이미 연결돼 있었다면 연결을 우클릭해 **새로 고침(F5)** 하세요. API는 `http://localhost:8000/docs`에서 확인할 수 있습니다.

4. 복원한 뒤

4-1. 데이터를 최신으로 유지하는 방법

방법	어떻게	특징
A. 그대로 쓰기	아무것도 안 함	덤프 시점에 멈춘 데이터. 화면·API 개발에는 충분합니다
B. 새 덤프 다시 받기	3장을 다시 실행	가장 간단합니다. 기존 DB는 통째로 바뀝니다
C. 내 PC에서 수집 켜기	`backend/.env`에 API 키 입력 후 backend를 켜 두기	매일 오전 6시에 자동 수집. **처음부터가 아니라 덤프 이후 새 공고·새 달만** 이어서 받습니다

4-2. 수집을 켜려면 (방법 C)

.env 항목	필요한 것
`APPLYHOME_API_KEY`	공공데이터포털 일반 인증키. 계정에서 아래 API를 **각각 활용신청**해야 합니다 (신청 전에는 401·403, 그 부분만 건너뜀) · 청약홈 분양정보 조회 서비스 · 청약홈 청약 경쟁률 조회 서비스 · 청약홈 청약통장 통계 조회 서비스 · 청약홈 청약 신청·당첨자 정보 조회 서비스 · LH 분양임대공고 조회 · 공고별 상세정보 조회 · 공고별 공급정보 조회
`LH_API_KEY`	LH API를 다른 계정에서 승인받았을 때만. 비어 있으면 위 키를 씁니다
`FINE_API_KEY`	금융감독원 금융상품통합비교공시(파인) 인증키

```
# 수동으로 한 번 수집 (덤프 이후 변경분만 받아서 몇 분 ~ 수십 분)
docker compose exec backend python -m app.data_pipeline.save_listings

# 자동 수집: backend 컨테이너가 켜져 있으면 매일 06:00 (Asia/Seoul)에 실행됩니다
```

> **같은 키를 여러 명이 쓰면** LH API의 하루 10,000건 한도를 나눠 쓰게 됩니다. 수집은 한 사람(또는 공용 서버)만 돌리고 나머지는 덤프를 받는 방식이 안전합니다.

4-3. 코드가 바뀌어 테이블 구조가 달라졌을 때

마이그레이션 도구가 아직 없어서, 새 코드가 컬럼을 추가했는데 내 DB가 예전 구조면 오류가 납니다 (`column … does not exist`). 이때는 **새 코드와 짝이 맞는 새 덤프를 받아 3장을 다시 실행**하는 것이 가장 쉽습니다.

5. 자주 생기는 문제

증상 / 오류 메시지	원인	해결
`could not open input file "C:/…/cheongyak.dump"` `could not open output file "C:/…"`	Git Bash가 `/tmp/…` 경로를 윈도우 경로로 바꿈	**PowerShell에서 실행**하세요. Git Bash를 써야 하면 명령어 앞에 `MSYS_NO_PATHCONV=1` 을 붙입니다
`database "cheongyak" is being accessed by other users`	backend나 DBeaver가 DB에 연결 중	`docker compose stop backend` 후 다시. 가이드의 `WITH (FORCE)` 가 있으면 연결을 끊고 지웁니다
`relation "…" already exists` 가 줄줄이 나옴	③(DB 지우고 새로 만들기)을 건너뛰고 복원	③부터 다시 실행
`service "db" is not running`	DB 컨테이너가 꺼져 있음	`docker compose up -d db`
`no configuration file provided: not found`	프로젝트 폴더가 아닌 곳에서 실행	`docker-compose.yml` 이 있는 폴더로 이동 (`cd …\cheongyak-platform`)
`unsupported version (1.xx) in file header`	덤프를 만든 PostgreSQL이 더 최신 버전	이 프로젝트는 `postgres:16` 고정이라 보통 생기지 않습니다. `docker compose pull db` 후 다시
복원은 됐는데 backend가 `column … does not exist`	코드와 덤프의 날짜가 안 맞음	`git pull` 로 덤프와 짝이 맞는 코드를 받고 `docker compose up -d --build backend`
backend가 `ModuleNotFoundError: pdfplumber / openpyxl`	이미지를 다시 빌드하지 않음	`docker compose up -d --build backend`
DBeaver에 예전 테이블만 보임	화면이 갱신되지 않음	연결 우클릭 → 연결 해제 후 다시 연결, 또는 F5

한 번에 보는 요약

보내는 사람

① `pg_dump -Fc` 로 만들기 (15초)
② `docker compose cp` 로 꺼내기
③ 드라이브로 전달 + 짝이 맞는 브랜치·날짜 알리기

받는 사람

① `git pull` · backend 멈춤
② 덤프를 컨테이너에 복사
③ DB 지우고 새로 만들기 → `pg_restore` (1분)
④ backend `--build` 로 다시 켜고 행 수 확인

명령어는 PostgreSQL 16.15 (Docker `postgres:16`), Windows 11 PowerShell에서 실행해 확인했습니다: 2026-10-02 덤프 15초·62.7MB, 복원 42초, 복원 후 16개 테이블·뷰의 행 수가 원본과 일치.